

从“拉群救火”到“AI 排查”： 星图端到端智能诊断体系的工程实践

宁振航

小红书/技术专家



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 背景与思考

02 端到端智能诊断的核心挑战

03 大模型驱动的问题排查体系构建

04 能力框架与落地实现路径

05 总结与展望

01

背景与思考

研发侧的痛点与破局思考

研发侧的痛点——高频问题消耗大量资源

⚠ 现象

日常工作中，研发同学经常被拉到问题处理群以及收到工单要进行处理，线上问题排查已成为研发团队的日常高频工作

问题量级

数百个/周

每周流转至研发值班同学

人力投入

数十人

需多名研发同学轮值支持

📊 问题分布拆解

55%

系统类问题

5-30 分钟

登录失败、接口异常、数据不一致

25%

策略类问题

5-30 分钟

笔记未展示、活动未命中、效果未达预期

20%

咨询类问题

3-10 分钟

服务归属、接口用法、逻辑说明

💡 **核心洞察：**这种高频的中断，让研发团队在需求交付之外承担了极重的运维包袱，严重拖垮了整体的研发能效



核心矛盾与破局思考——低价值重复工作占比过高

核心矛盾数据

问题需研发介入

必须由懂代码的研发同学才能查清

70%

重复老问题

已有明确排查路径和解决方案

40%

⚠ 这意味着我们正在用 **最高昂的研发成本**，去处理大量“已知路径的重复诊断”工作。

破局思考

资深研发排查流程：



现象感知



线索收集



根因归因



修复建议

目标：将专家经验转化为 AI Agent 推理链

- ✓ 减少重复排查 → 常见问题自动化诊断
- ✓ 降低值班投入 → 释放人力聚焦核心开发
- ✓ 提升交付效率 → 响应从“小时级”降至“分钟级”

🔑 核心目标：实现 **研发专家经验的规模化复制**，让AI成为承载排查智慧的执行体。

02

端到端智能诊断的核心挑战

挑战一：三端观测平台各自为政

客户端

监控工具

埋点系统、性能监控

排查思路

页面行为、埋点数据

关注指标

崩溃率、页面加载时长

前端

监控工具

Sentry、前端监控平台

排查思路

接口请求、渲染链路

关注指标

JS错误率、API响应时间

服务端

监控工具

Grafana、ClickHouse、Trace

排查思路

日志、配置、调用链

关注指标

QPS、错误率、P99延迟

核心痛点

- 工具差异：不同端依赖完全不同的工具链（Grafana、Sentry、埋点系统等）
- 排查路径：端侧关注容器，前端关注渲染逻辑，服务端关注日志链路
- 知识形态：代码、监控、日志、文档、配置、历史工单等多种信息源混杂

为什么不能“大一统”？

若采用固定的诊断流程，必然僵化，难以适应不同端对观测平台的特化需求

- AI需要具备跨端自适应能力，而非单一固定模式

挑战二：日志噪音淹没关键信息

数据量级问题

单次请求日志

单服务场景

数百条

跨服务场景

微服务架构下完整调用链

数千条

噪音干扰类型



接口日志

接口的请求信息，线上无价值有限



方法执行流程日志

打印方法调用链路与执行步骤



框架/中间件日志

Spring、Task、MQ等框架自动输出的运行状态日志



上下文超限：Token瓶颈

大模型存在 **Token限制**，无法直接消费原始日志的全量内容。海量噪音不仅掩盖了核心错误，更会 **直接撑爆大模型的Token上下文窗口**，导致模型"失忆"或报错。

典型模型上下文

128K-256K

原始日志Token

10K-500K

超出比例

1-4x

🔑 **核心难题**：如何在海量日志中 **精准定位"与问题相关"的关键片段**，是上下文工程的首要难题。



挑战三：日志与代码脱节

📖 日志能告诉我们什么？

示例日志：

```
2025-04-17 14:32:15
ERROR login check failed:
  invalid credentials
  uid=12345
```

- ✓ 发生了什么（登录认证失败）
- ✓ 错误信息（用户或者密码失效）
- ✓ 关联对象（用户ID）

❓ 日志无法告诉我们什么？



代码执行路径

代码执行逻辑是什么样子？经过了哪些if-else分支？



条件判断逻辑

进入该分支的前置条件是什么？



局部变量状态

打印时的变量值、对象状态

👤 资深研发的人工排查模式



⚠️ **核心瓶颈：**无法将日志与打印它的代码上下文关联起来，就难以实现真正的“**根因归因**”。

挑战四：问题归因入口难定位

用户反馈的业务语言

“ 示例反馈

”账号登录失败了，收不到手机验证码“

“ 示例反馈

”账号关注数据展示不对，粉丝数量少了“

“ 示例反馈

”账号申诉点击提交没反应“

i 用户只会描述**业务现象**，而非技术路径

技术层面的复杂性

仓库众多

同一个功能可能横跨客户端仓库、BFF层仓库、多个后端服务仓库

入口多元

Web接口、定时任务、消息消费、RPC调用等多种入口，每种对应不同仓库

调用链复杂

一次用户操作可能触发跨服务的多次内部调用，问题可能发生在任意环节

核心瓶颈：冷启动难题

面对三端数十个代码仓库，**仅凭问题描述难以快速锁定“应该去哪个仓库看代码”**。这是AI诊断的“冷启动”难题——如果连起点都不知道，诊断根本无从发起。

代码仓库

数百个

服务数量

上千个

入口类型

10+

挑战五：效果评测黑盒化

模型不可解释性

🔄 **输出随机性** 相同输入可能产生不同输出，难以复现和预测

👻 **潜在幻觉** 可能编造不存在的代码分支、变量或调用链

❓ **推理不透明** 无法解释为什么会得出某个结论

工程化演进的基建缺失

❓ **无法量化**

如何科学证明新版本的Prompt或Agent调度逻辑比老版本更好？**缺乏客观评估标准**

排查场景的极高要求



准确率要求极高

错误的诊断结论比没有结论更可怕

- ✖ **误导风险**：错误结论会严重误导研发排查方向，浪费大量时间
- ✖ **信任危机**：一次错误可能摧毁用户对AI诊断的信任
- ✖ **业务影响**：问题排查延迟可能导致业务损失

\$ **回归成本极高**

传统依靠人工Review几百个历史工单来验证效果，**成本不可接受且极易漏测**

💡 **核心结论**：缺乏**自动化评测和实验发布机制**，大模型应用将无法进行安全、持续的演进。

03

大模型驱动的问题排查体系构建

核心架构全景图：三层架构设计



知识库前置路由层

核心任务
解决"去哪查"

- ✓ 业务语言→代码入口
- ✓ 仓库定位与上下文
- ✓ 观测平台入口映射



Coding Agent 深度诊断层

核心任务
解决"怎么查"

- ✓ 自由循环推理
- ✓ Skill自动触发
- ✓ 代码深度下钻



评测与实验发布底座

核心任务
解决"敢不敢发"

- ✓ 数据观测飞轮
- ✓ LLM as Judge
- ✓ 分层指标与安全发版

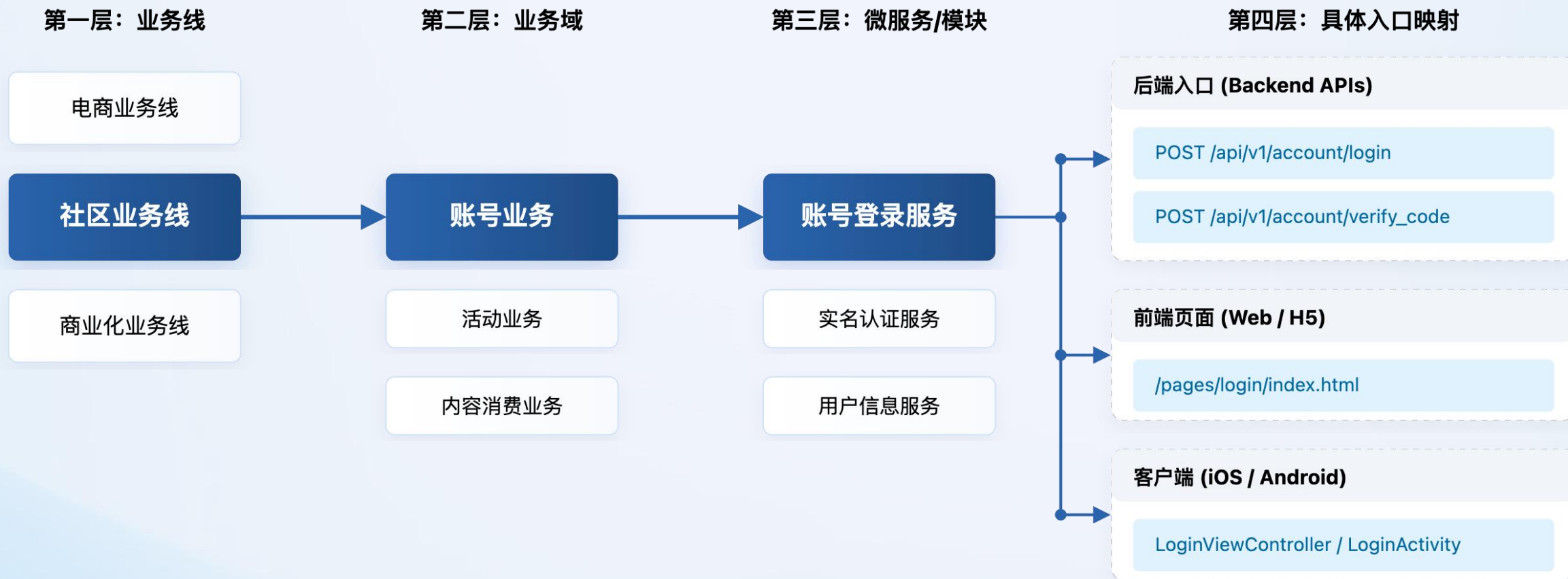


架构设计理念

上层通过**AI知识库**实现问题场景到代码/平台入口的快速定位，中层通过**Coding Agent**完成深度诊断，底层通过**评测与实验发布体系**保障能力的安全迭代。



知识库前置路由——解决反馈与逻辑的关联问题



基于“渐进式披露”的前置路由策略设计，根据用户的意图逐层下钻，逐步缩小问题域范围，避免context过载，精准定位上下文



路由地图内容的生成机制：AI + 人工双轨制



AI自动维护
80%



代码注释解析

提取JavaDoc/GoDoc/Swagger注解中的语义信息



调用链自动发现

基于代码链路分析+运行时Trace，构建接口级依赖图谱



文档同步更新

代码变更后自动触发知识库增量更新，保持与代码库同步



人工补充
20%



平台特化限制

如"某端不支持繁体字""特定字段有长度限制"等隐性规则



链路逻辑说明

复杂策略的业务背景、历史设计决策、特殊case处理



排查经验沉淀

如"遇到错误码X先看Y服务状态""该问题需先查客户端埋点"



双轨制工作机制

AI自动生成内容占80%，实现 **低成本广覆盖**；人工补充占20%，沉淀 **高价值经验**。人工补充内容自动与AI生成内容合并，形成完整的接口知识卡片

AI生成

80%

自动化、规模化

人工补充

20%

高价值、经验性

合并输出

100%

完整知识卡片

Coding Agent: 深度下钻的核心流程

Agent诊断漏斗



</> 代码分析

Agent接管诊断流程后，首先读取Controller/Service代码，识别关键逻辑节点

- > 外部服务调用 (RPC/HTTP)
- > 异常处理逻辑 (try-catch)
- > 配置读取 (配置中心)
- > 日志输出点 (logger.info)

✖ 自动触发Skill

基于代码分析结果，自动触发对应的诊断Skill 获取证据：

-  LogQuerySkill → 查询日志
-  MetricSkill → 查询监控
-  TraceSkill → 查看链路
-  TicketSkill → 查询工单

Coding Agent: 自由循环架构 (Free Loop)

三 流式推理映射

核心机制：



长文本推理

大模型思考过程可达数千token



实时提取决策

流式解析，实时提取关键决策点



立即转为Skill调用

如"需要查看预算服务配置项"

- ✓ 确保模型思考完整后再行动，避免因推理片段丢失导致的误判

自驱动停止



不预设固定迭代次数

传统方式：固定循环3轮或5轮，简单问题浪费，复杂问题不够



模型自主决定

当模型输出纯文本结论、不再请求调用Skill时，任务自然终止

实际表现：

3-5轮

简单问题

20+轮

复杂问题



自由循环的核心优势

排查问题是一个**"且战且走"**的过程。自由循环架构让Agent像资深研发一样，根据当前证据自主决定下一步行动，而非被固定流程束缚。简单问题快速收敛，复杂问题可以持续深挖，直到找到确凿证据。

真实Case全链路拆解



场景

用户反馈"实名认证提交后状态一直显示认证中，超过30分钟仍未通过"

1 知识库路由

匹配场景
用户身份实名认证链路

定位仓库
user-identity-service

提取上下文
认证状态由第三方实名核验服务异步回调更新

2 深度下钻

</> 查代码：调用外部实名核验服务API

📖 查日志：回调请求响应异常，大量返回错误

🔗 查Trace：核验API P99超5秒，大量超时

📺 查监控：发现核验服务成功率下跌

3 诊断结论

根因 核验服务系统故障导致实名认证失败

建议 已联系外部服务负责人，建议用户稍后重试

关联工单 REDTICKET-2024-12345



诊断效率对比

👤 人工排查

耗时

跨系统

经验依赖

30+分钟

需人工切换多个平台
高度依赖资深研发

🤖 AI Agent排查

耗时

自动化

一致性

5分钟

全自动跨系统查询
标准化流程输出

04

核心问题解决与落地

上下文工程：应对大模型的Token瓶颈

🔍 动态探索项目状态

为什么不依赖静态向量库？

- ✗ 线上代码和状态变化太快，向量库容易过时
- ✗ 无法反映未提交的本地变更
- ✗ 增量更新成本高，维护困难

Agent主动探索能力

- ✓ 读取 package.json / pom.xml / go.mod 了解项目结构
- ✓ 扫描目录识别项目类型
- ✓ 查看Git历史了解最近变更
- ✓ 检查未提交变更避免冲突

✂️ 信息压缩策略

P1 Error / 异常堆栈

完整保留

关键错误信息一字不落，确保根因定位的准确性

P2 Warn 日志

分析提取

大模型分析并提取关键Key-Value对

P3 Info 日志

生成摘要

超长智能裁剪，保留关键上下文信息

🎯 上下文工程的核心目标

在不丢失核心证据的前提下，**智能控制Token消耗**。当上下文接近模型窗口上限时，采用智能摘要保留关键信息，而非简单截断，确保诊断的完整性和准确性。



诊断流程的宏观编排

漏斗式编排架构



问题接入



意图识别

轻量分类模型



上下文收集

并行执行



线索分析

自由循环



根因推断

LLM推理



结论输出



意图识别前置

通过 **轻量模型快速分类** 问题类型，动态选择Skill组合：

端侧异常

前端异常

服务端异常

策略问题

咨询问题

💡 识别问题排查需要加载的SKILL，并给出优先排查的方向



人机协同设计



转人工入口

复杂问题提供"转人工"入口，附带已收集的证据



危险操作拦截

敏感操作前暂停等待确认



死循环保护

尝试多次无头绪自动终止并转人工

评测与实验底座——自动化观测飞轮

数据流转闭环飞轮



线上埋点采集

- 完整记录用户排查意图
- Agent推理轨迹(Trace)
- 多轮会话日志



Badcase自动挖掘

- 用户评价(点踩)
- 审计拦截失败案例
- 自动归类(幻觉/漏看)



沉淀Benchmark

- 拆解Badcase
- 转化为测试用例
- 版本化管理



回归驱动升级

- 跑通LLM自动化回归
- 验证Prompt/架构调整
- 驱动架构迭代

自进化飞轮

线上采集 → Badcase治理



补充Benchmark ← 架构迭代



LLM自动化回归

平台的每一次交互都为下一次升级积累数据，形成从线上到线下的完整闭环流转

Badcase自动归类



幻觉影响

编造不存在的代码或配置



关键日志漏看

未查询重要日志线索



根因偏离

诊断结论偏离真实根因

解决人工评测瓶颈——LLM as Judge

🎓 专家规范对齐

并非让裁判自由发挥，而是通过Few-shot将资深研发的诊断规范注入模型：

- ✓ 定义标准排查流程和检查点
- ✓ 提供典型正确/错误案例
- ✓ 明确评分维度和权重

💡 关键经验

我们的做法是设计“原子化倒扣分”规则

🔄 COT思维链打分

必须基于思维链输出详尽的扣分依据，再给出最终分数

- ✓ 增强可解释性，便于人工复核

📊 原子化倒扣分

未调取上游日志

裁判检查推理过程是否漏查关键日志

-10分

捏造不存在的缓存键

幻觉行为，编造代码中不存在的变量

-20分

未检查配置中心

代码读取配置但未查询配置值

-5分

📊 人机对齐验证

Kappa系数

机器评测与人工打分的一致性

90%+

通过人工定期抽检裁判结果并进行归因分析，实现AI替代人工规模化评测

业务结果落地与数据收益展现

平台上线后核心业务数据

端到端问题排查覆盖率 **100%**

问题排查结果准确率 **80%**

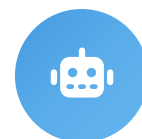
周问题排查数量 **1000+**

平均诊断耗时



人工

30分钟



AI

5分钟

耗时降低 **80%**

研发值班人效

+60%

人效提升

释放出大量时间聚焦核心需求迭代

05

总结与展望



总结与展望



关键工程经验



知识库是核心资产

AI自动生成(80%) + 人工补充(20%)的双轨制，确保知识库低成本广覆盖的同时沉淀高价值经验。代码变更自动触发知识库增量更新，保持与代码库同步



评测体系是演进基石

LLM as Judge + 原子化倒扣分机制实现自动化评测，Kappa系数达90%+。线上埋点采集→Badcase自动挖掘→Benchmark沉淀→回归驱动升级的完整飞轮



未来演进方向



自主学习

从人工补充到基于准确结论的自我学习，大幅提升诊断准确率



多Agent协同

构建多Agent架构，实现跨领域问题的协同诊断与知识共享



主动预防

从被动诊断演进为主动预警，基于模式识别提前发现潜在风险



人机协同

升级AI与研发的协作模式，让AI成为真正的智能助手

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

